



JENSEN Education

Yrkeshögskolan

When Does Kubernetes Become Worth It?

*Measuring Reliability, Stability, and Execution Overhead in Workflow Orchestration
Systems Moving from Shared VM to Isolated Kubernetes Execution*

Programme: DevOps24M

Term: Vårterminen 2026

Author: Sirajulhaq Wahaj

Supervisor: Ludvig Malm

Sammanfattning

This thesis investigates how reliability, system stability, and execution performance change when a workflow orchestration system migrates from a single virtual machine using the Dagster DockerRunLauncher to a Kubernetes cluster using the K8sRunLauncher with isolated pod execution. Dagster 1.12.22 is used as a representative modern pipeline orchestration framework, and the workload is a CPU-bound, memory-intensive two-phase job. Three controlled experiments were conducted across six concurrency levels (1, 2, 3, 5, 7, and 10 concurrent jobs, three repetitions each) on local infrastructure: a 4 vCPU / 4 GB Multipass VM running the DockerRunLauncher and a Kind single-node cluster running the K8sRunLauncher.

The VM's job success rate degraded sharply with concurrency: 100 % at L1–L2, falling to 66.7 % at L3, 46.7 % at L5, 42.9 % at L7, and 30.0 % at L10. The failure mechanism was memory exhaustion: three or more concurrent containers each allocating 400 MB exceeded the kernel's memory budget on the 4 GB VM, triggering Linux OOM kills. Execution time for surviving jobs remained relatively flat (65–80 s) across all levels, demonstrating that OOM-kill acts as a filter rather than a performance degradation mechanism. Kubernetes maintained 100 % success across all concurrency levels via per-pod cgroup limits, with consistent execution time and low variance. Pod scheduling latency was under 1 second; container startup overhead ranged from 4.3 s at L1 to 14.9 s at L6 in the local Kind environment with pre-pulled images, growing with concurrency due to concurrent initialisation contention. The reliability crossover — the first concurrency level where VM success rate drops below 95 % while Kubernetes maintains 100 % — occurred at **L3 (3 concurrent jobs)**. These findings give engineering teams an empirical, hardware-specific framework for migration decisions.

Keywords: Kubernetes, Dagster, workflow orchestration, resource contention, pod isolation, crossover point, Kind, reliability measurement, DevOps

Contents

Sammanfattning	i
1 Introduction	1
1.1 Background and Motivation	1
1.2 The Research Problem	2
1.2.1 The Surface Problem	2
1.2.2 The Deeper Problem — The Research Gap	2
1.2.3 The Literature Gap Statement	3
1.3 Purpose	3
1.4 The Core Idea	3
1.5 Primary Contribution — The Crossover Point	3
1.6 Research Hypothesis	4
1.7 Research Questions	4
1.8 How the Questions Form One Argument	5
1.9 Practical Value and Target Audience	5
1.10 Delimitations	5
1.11 Thesis Structure	6
2 Literature Review	7
2.1 Pillar 1 — Resource Contention and Non-Linear Performance Degrada- tion	7
2.2 Pillar 2 — Kubernetes Performance Overhead and Fault Isolation . .	8
2.2.1 Scheduling and Startup Overhead	8
2.2.2 Fault Isolation and Blast Radius	9
2.3 Pillar 3 — Autoscaling Behaviour and Response Latency	9
2.4 Positioning Statement — The Gap This Thesis Fills	10
3 Method and Experimental Design	11
3.1 Research Design	11
3.2 Infrastructure Strategy — Local-First with Optional Cloud Validation	11
3.2.1 Primary Environment: Local Infrastructure	11
3.2.2 Optional Cloud Validation Phase	12
3.2.3 Infrastructure Specifications	13
3.3 Tool Stack	13
3.3.1 Infrastructure Tools	13

3.3.2	Application and Orchestration	14
3.3.3	Measurement and Observability	14
3.3.4	Development and Documentation	14
3.4	Operational Definitions	14
3.5	Workload Definition	15
3.5.1	Workload Job Design	15
3.5.2	Concurrency Levels	16
3.5.3	Workload Rationale	16
3.6	Test Harness	17
3.7	Experimental Design	17
3.7.1	Experiment 1 — VM Degradation Profile (answers SQ1) . . .	18
3.7.2	Experiment 2 — Kubernetes Isolation and Blast Radius (an- swers SQ2)	18
3.7.3	Experiment 3 — Overhead Measurement and Crossover Point (answers SQ3, SQ4)	19
3.8	Metrics Summary	20
3.9	Validity Statement	20
4	Results	21
4.1	Experiment 1 Results — VM Degradation Profile (SQ1)	21
4.1.1	Observed Degradation Pattern	22
4.1.2	Resource Utilisation	23
4.2	Experiment 2 Results — Kubernetes Isolation (SQ2)	24
4.2.1	Part A — Isolation Comparison	24
4.2.2	Part B — Blast Radius Test	27
4.2.3	Part C — Scheduling Overhead (SQ3 Supplementary)	27
4.3	Experiment 3 Results — Overhead and Crossover Point (SQ3, SQ4) .	28
4.3.1	Kubernetes Scheduling Overhead (SQ3)	28
4.3.2	The Crossover Point (SQ4)	29
5	Discussion	32
5.1	Discussion of SQ1 — The VM Failure Threshold	32
5.2	Discussion of SQ2 — Pod Isolation Effectiveness	33
5.3	Discussion of SQ3 — Kubernetes Scheduling Overhead	34
5.4	Discussion of SQ4 — The Crossover Point	36
5.5	Trade-off Analysis	38
5.5.1	What Kubernetes Improves	38
5.5.2	What Kubernetes Costs	39
5.5.3	Decision Framework	39
5.6	Limitations	40

6	Conclusions	42
6.1	Summary of Findings	42
6.2	Answer to the Main Research Question	43
6.3	Practical Recommendations	44
6.4	Future Research	45
	References	47
	Appendices	48
A	Kind Cluster Configuration	49
B	Dagster Helm Values (Kubernetes Run Launcher)	50
C	Test Harness Pseudocode	51
D	Experiment Data Collection Template	53

List of Figures

4.1	VM success rate and mean execution time vs. concurrency level. Success rate collapses from 100 % at L1–L2 to 30 % at L10, driven by OOM kills during the memory pressure phase. Execution time of <i>successful</i> jobs remains comparatively stable (65–80 s).	22
4.2	Kubernetes experiment dashboard: success rate, execution time distribution, startup overhead, and total time per job vs. concurrency level (Exp2A, $n = 3$ reps). Success rate remains 100% at all levels; execution time grows modestly from 63 s to 82 s, reflecting effective pod-level resource isolation.	25
4.3	Kubernetes job success rate by concurrency level (Exp2A, $n = 3$ reps per level). All six levels maintained 100 % success, well above the 95 % reliability threshold.	25
4.4	Kubernetes job execution time distribution by concurrency level (Exp2A, $n = 3$ reps). Boxes show interquartile range; whiskers extend to $1.5\times$ IQR. The distribution widens at L7 and L10 as node-level memory pressure increases, but remains tightly bounded at lower levels.	26
4.5	Execution time comparison: VM vs. Kubernetes across concurrency levels. The VM curve rises steeply due to CPU contention; the Kubernetes curve remains nearly flat due to pod isolation. The crossover point — where K8s begins outperforming the VM — is identified in Section 4.3.	26
4.6	Kubernetes scheduling overhead analysis (SQ3). Left: total startup overhead per concurrency level (mean $\pm$ std across 3 reps). Right: overhead component breakdown (pod scheduling latency, container startup, Dagster job init). Overhead grows from 4.3 s at L1 to 14.9 s at L10 due to concurrent initialisation contention on the single-node Kind cluster.	29
4.7	Crossover chart: VM job success rate and execution time vs. Kubernetes total time (execution + startup overhead) across concurrency levels. The reliability crossover (VM below 95 %) occurs at L3; the performance crossover (K8s total time below VM survivor time) occurs at L2–L5.	30

List of Tables

3.1	Infrastructure comparison between VM and Kubernetes environments.	13
3.2	Infrastructure tools used in this thesis.	13
3.3	Application and orchestration tools.	14
3.4	Measurement and observability tools.	14
3.5	Development and documentation tools.	14
3.6	Concurrency levels and workload profile. All levels use the same per-job resource footprint.	16
3.7	Complete metrics table.	20
4.1	VM deployment results by workload level (3 repetitions, 4vCPU / 4GB RAM). Mean time and std are for <i>successful</i> runs only. CPU and memory are system-wide averages during the run window.	21
4.2	Kubernetes deployment results by workload level (mean $\pm$ std over 3 repetitions, Kind single-node cluster, 4vCPU / 8GB node, 2GiB per- pod memory limit). All concurrency levels maintained 100% success rate under stable node-level conditions.	24
4.3	Blast radius comparison at L5 (5 concurrent jobs) — deliberate single- job failure injection.	27
4.4	Kubernetes scheduling overhead by concurrency level. Values are means over 3 repetitions $\times k$ concurrent pods per level. Startup overhead = total pod lifetime – Dagster execution time (joined on <code>run_id</code>).	28
4.5	Net execution time comparison: VM vs. Kubernetes. VM mean is for <i>surviving</i> jobs only. K8s total = Dagster execution time + startup overhead (pod lifetime – exec time, joined on <code>run_id</code>). Δ = K8s total – VM mean: negative means K8s is faster. At L3+ the VM success rate falls below 95%, making raw speed comparison misleading.	29
D.1	Raw data collection template for experimental runs (VM environment, one repetition shown; repeat for reps 2 and 3 and for K8s environment). Pod scheduling and container startup columns are K8s-only; enter “—” for VM rows.	53

Chapter 1

Introduction

1.1 Background and Motivation

Modern data engineering relies on pipeline orchestration frameworks to automate, schedule, and monitor workflows across unpredictable workloads. A common initial deployment strategy is to host such a framework on a single virtual machine (VM) using a Docker-based run launcher, where each pipeline job runs in its own container on the same host. While simpler than Kubernetes, this architecture carries a fundamental limitation: all containers share the host’s finite pool of CPU and memory, and the Linux kernel’s OOM killer operates at the host level rather than being scoped to individual containers.

As concurrent job execution increases, resource contention arises. Jobs compete for the same resources, performance degrades non-linearly, and under sufficient load the system fails entirely. Research on shared-memory multiprocessor systems demonstrates that this degradation compounds multiplicatively rather than additively as utilisation approaches capacity limits, producing a collapse effect rather than a smooth performance curve (Nanda et al., 1991). Arora (2023) further establishes that shared resource contention influences task timing in non-deterministic ways, making worst-case execution behaviour extremely difficult to predict.

Dagster is used in this thesis as a representative modern workflow orchestration system. It provides a Kubernetes Run Launcher that dispatches each pipeline run as an isolated pod on a Kubernetes cluster, where each pod receives defined CPU and memory resources. Failures are theoretically contained within individual pods rather than propagating across the system.

While this architectural migration is well-documented at the tool level, there is limited empirical evaluation of its actual effectiveness under controlled workload conditions. Existing research evaluates Kubernetes performance at the infrastructure level—measuring container overhead, pod scheduling latency, and autoscaler response times (Casalicchio, 2019; Choi et al., 2021; Nguyen et al., 2020). Separately, resource contention theory predicts non-linear collapse in shared execution environments

(Arora, 2023; Nanda et al., 1991). But no study connects these two bodies of evidence at the application-level workflow execution layer to answer the practical question: *at what workload level does migrating from a shared VM to Kubernetes become worth it?*

1.2 The Research Problem

1.2.1 The Surface Problem

A VM-hosted workflow orchestration system becomes unreliable under concurrent workload because all jobs share the same compute resources. This is an observable and known failure mode.

1.2.2 The Deeper Problem — The Research Gap

The surface problem is not the research gap. The gap exists at the intersection of two bodies of literature that have not yet been connected.

Existing research evaluates Kubernetes performance and autoscaling behaviour at the infrastructure level—measuring container overhead, pod scheduling latency, and cluster autoscaler response under generic workloads (Casalicchio, 2019; Nguyen et al., 2020; Tran et al., 2022). Separately, research on resource contention analyses performance degradation in shared execution environments (Arora, 2023; Nanda et al., 1991). What is missing is empirical measurement of how infrastructure-level Kubernetes mechanisms—pod isolation, resource limits, and horizontal autoscaling—actually affect application-level workflow execution behaviour under controlled concurrent workload conditions.

Casalicchio (2019) demonstrates that under concurrent workloads with CPU utilisation above 75%, response times under the default Kubernetes Horizontal Pod Autoscaler are 2 to 3 orders of magnitude higher than under a contention-aware algorithm, highlighting that resource contention remains severe even in containerised environments. Choi et al. (2021) report that the median container startup latency in production orchestration systems is 25 seconds, representing a substantial fixed overhead for every pipeline job. But the specific workload threshold at which migrating from VM-based to Kubernetes-based workflow execution changes the balance between contention cost and scheduling overhead has not been empirically identified.

This thesis bridges that gap by measuring system behaviour at the pipeline execution level rather than at the infrastructure level alone.

1.2.3 The Literature Gap Statement

Existing studies focus on Kubernetes performance and autoscaling at the infrastructure level, but there is no empirical evidence on how these mechanisms affect application-level workflow orchestration systems under controlled concurrent workload conditions. No study identifies the specific concurrency threshold at which a Kubernetes migration becomes net beneficial for pipeline orchestration. This thesis addresses that specific gap.

1.3 Purpose

The purpose of this thesis is to design, implement, and evaluate a controlled experimental comparison of single-VM and Kubernetes-based Dagster pipeline execution, measuring how reliability, system stability, and execution performance change under increasing concurrent workload, and identifying the specific workload threshold at which the Kubernetes migration becomes net beneficial.

1.4 The Core Idea

This thesis measures how reliability, failure behaviour, execution performance, and system stability change under increasing concurrent workload when a workflow orchestration system moves from a shared execution model on a single VM to an isolated distributed model on Kubernetes. It identifies the specific workload level at which that migration becomes net beneficial.

This thesis is **not** a proof that Kubernetes is better than a VM. It is not a Kubernetes benchmark, a tool comparison, or a cloud provider comparison. It focuses specifically on behavioural differences between two architectural execution models—VM-hosted Docker containers (DockerRunLauncher) versus Kubernetes-managed pods (K8sRunLauncher)—measured at the workflow job execution level under controlled conditions. If results show the VM performs better across all tested concurrency levels, that is an equally valid and valuable finding.

1.5 Primary Contribution — The Crossover Point

The primary contribution of this thesis is identifying and measuring a crossover point, defined through two complementary thresholds:

Reliability crossover

The concurrent workload level at which the VM deployment’s job success rate

drops below 95% due to resource contention.

Performance crossover

The concurrent workload level at which the Kubernetes deployment’s total execution time (including pod scheduling overhead and container startup latency) becomes lower than the VM’s execution time under active contention.

These two thresholds may occur at different concurrency levels. If so, both are reported alongside a **combined crossover point**—the level at which both conditions are simultaneously met. This dual definition avoids missing cases where the VM is already severely degraded in performance but has not yet crossed the success rate threshold.

1.6 Research Hypothesis

Migrating workflow orchestration execution from a single-VM DockerRunLauncher to a Kubernetes Run Launcher will produce measurable reliability improvements through pod-level resource isolation, but will also introduce measurable scheduling latency and execution overhead that does not exist in the VM deployment.

These advantages are conditional on workload level. At low concurrency, the VM is expected to outperform Kubernetes on execution speed because resource contention has not yet reached critical levels and there is no pod scheduling overhead to pay. At high concurrency, Kubernetes is expected to outperform the VM as pod isolation prevents the contention collapse that a shared execution model cannot avoid.

1.7 Research Questions

Main Research Question

How do reliability, system stability, and execution performance change when a workflow orchestration system is migrated from a single-VM DockerRunLauncher to a Kubernetes Run Launcher under increasing concurrent workload, and at what specific workload level does this migration become net beneficial?

Supporting Questions

SQ1 — VM Failure Threshold and Degradation Profile

At what concurrency level does a single-VM workflow deployment fail, and how

do job success rate, execution time variance, and resource utilisation degrade as concurrent load increases toward that threshold?

SQ2 — Pod Isolation Effectiveness and Blast Radius Containment

To what degree does Kubernetes pod isolation contain failures and reduce execution time variance compared to the single-VM deployment under equivalent concurrent workload levels?

SQ3 — Kubernetes Scheduling Overhead

What measurable scheduling latency and execution overhead does Kubernetes introduce compared to the VM DockerRunLauncher at each concurrency level?

SQ4 — The Crossover Point

At what concurrency level does the Kubernetes overhead become smaller than the VM's contention-induced degradation—the formally defined crossover point?

1.8 How the Questions Form One Argument

The four supporting questions form a single connected logical chain:

- **SQ1** establishes the exact failure profile of the VM architecture—the baseline evidence.
- **SQ2** tests whether Kubernetes actually fixes what SQ1 measured as broken.
- **SQ3** quantifies the cost of the Kubernetes execution model—the overhead that must be justified.
- **SQ4** synthesises SQ1 through SQ3 into the crossover point—the primary contribution.

1.9 Practical Value and Target Audience

These findings give small teams, startups, and engineers a data-based answer to: *at what workload scale does migrating a workflow orchestration system from a VM to Kubernetes become worth the added complexity and overhead?*

1.10 Delimitations

This study does not include:

- Comparison between cloud providers (AWS EKS, Azure AKS)
- Comparison between orchestration tools—the focus is on execution model behaviour, not tool selection
- ML model accuracy evaluation
- Alternative VM executors—only the DockerRunLauncher is used on the VM (no CeleryK8sRunLauncher, no in-process execution)
- Real or sensitive data—synthetic workloads only
- Production-grade multi-node Kubernetes configurations (unless GCP budget permits the optional cloud validation)

Ethical statement. This study uses only synthetic workloads. No personal, sensitive, or proprietary data is collected or processed at any stage.

1.11 Thesis Structure

Chapter 2 reviews the literature across three pillars. Chapter 3 defines the method, infrastructure, experimental design, tools, and metrics. Chapter 4 presents the results of all three experiments. Chapter 5 discusses the findings in relation to the research questions and the crossover point. Chapter 6 presents conclusions, practical recommendations, and directions for future research.

Chapter 2

Literature Review

This chapter reviews three interconnected research areas that provide the theoretical foundation for the experimental design. Each pillar connects directly to one or more supporting questions. The chapter concludes with a positioning statement identifying the specific gap this thesis fills.

2.1 Pillar 1 — Resource Contention and Non-Linear Performance Degradation

Theoretical basis for SQ1.

Resource contention arises when multiple processes compete for finite shared resources—most commonly CPU cycles and memory—on a single host. The defining characteristic relevant to this thesis is that performance degradation under contention is *non-linear*: it does not scale proportionally with load but accelerates disproportionately as resource utilisation approaches capacity limits.

[Nanda et al. \(1991\)](#) provide the foundational empirical framework for this phenomenon in shared-memory multiprocessors. They introduce two normalised metrics—*efficiency* and *overhead factor*—to quantify the gap between ideal parallel speedup and actual performance when concurrent processes share memory resources. Their key finding is that contention-induced overhead compounds multiplicatively rather than additively, and can be decomposed into identifiable components: software synchronisation overhead, lock contention, and hardware memory access contention. This decomposition is directly relevant to understanding VM failure modes: when Dagster’s DockerRunLauncher runs concurrent jobs as Docker containers on the same host, the overhead includes OS-level container scheduling, shared host memory contention, and I/O channel arbitration.

[Arora \(2023\)](#) extends contention analysis to multicore processors, demonstrating that shared resource contention is inherently non-deterministic—it can influence the temporal behaviour of tasks in unpredictable ways. A task executing on one

core competes with co-running tasks for access to shared caches, memory bus, and main memory. The work shows that contention depends not only on the number of concurrent tasks but also on memory access patterns and arbitration policies. Critically, [Arora](#) demonstrates that contention effects can cause timing violations that cascade through dependent tasks—the same cascading failure mode seen when Docker containers on a shared VM host exhaust memory and the kernel OOM killer terminates multiple containers at once.

[Casalicchio \(2019\)](#) brings contention theory into the container domain, demonstrating that when multiple container instances run on the same host, they generate “interference in the contention of physical resources.” Under concurrent workloads with CPU utilisation above 75%, response times under the default Kubernetes Horizontal Pod Autoscaler (KHPA) are 2 to 3 orders of magnitude higher than under a contention-aware algorithm. This finding confirms that resource contention remains a severe problem even in containerised environments when containers share a host.

Research connection. SQ1 identifies the empirical concurrency threshold at which non-linear contention collapse occurs for Dagster pipeline workloads on a single VM. The literature predicts the collapse will be sudden rather than gradual.

2.2 Pillar 2 — Kubernetes Performance Overhead and Fault Isolation

Theoretical basis for SQ2 and SQ3.

This pillar combines two tightly related concerns: the overhead Kubernetes introduces and the fault isolation it provides. Both are trade-offs of the same architectural decision.

2.2.1 Scheduling and Startup Overhead

A substantial body of research documents the performance overhead of containerisation and Kubernetes orchestration. The primary sources of overhead relevant to this study are *pod scheduling latency*—the time between job submission and pod assignment to a node—and *container startup time*—the time between pod assignment and first application instruction. [Choi et al. \(2021\)](#) cite Google Borg data showing a median container startup latency of 25 seconds, with 90th-percentile microservice startup at approximately 15 seconds. [Liu et al. \(2024\)](#) report that scheduling latency has become a new bottleneck in Kubernetes-based systems, particularly for short-duration workloads where fixed overhead constitutes a large fraction of

total execution time. For a workflow orchestration system like Dagster using the Kubernetes Run Launcher, each pipeline run pays this overhead cost—a cost that is higher than the Docker container startup overhead in the VM DockerRunLauncher deployment.

2.2.2 Fault Isolation and Blast Radius

A foundational principle of distributed systems design is that failures should be contained within the smallest possible boundary. *Blast radius*—the scope of a system affected by a single failure—is central to reliability architecture (Čilić et al., 2023). In a single-VM shared execution environment, the blast radius of any individual job failure is potentially the entire system: a job that exhausts memory can trigger an out-of-memory kill that terminates all running jobs. In the Kubernetes pod model, each job runs with enforced resource limits, and a failing pod is terminated without directly affecting other pods. However, Casalicchio (2019) demonstrates that pod isolation is not absolute—containers on the same host still generate interference through physical resource contention even when they have defined resource limits.

Research connection. SQ2 empirically tests whether pod isolation contains failures at the workflow execution level. SQ3 quantifies the scheduling overhead at each concurrency level. Together, they provide the data needed to compute the crossover point (SQ4).

2.3 Pillar 3 — Autoscaling Behaviour and Response Latency

Contextual basis for understanding Kubernetes operational behaviour.

Kubernetes Horizontal Pod Autoscaler (HPA) scales pod count based on observed CPU or memory metrics, while the cluster autoscaler provisions new nodes when scheduling demand exceeds current capacity. Both mechanisms introduce reaction latency absent from a fixed-capacity VM deployment.

Nguyen et al. (2020) provide a comprehensive empirical analysis of HPA’s operational behaviour, revealing that scaling decisions are sensitive to metric scraping periods, cluster size, and the type of metric used—knowledge “not available on the official website and other sources.” Tran et al. (2022) survey the state of the art and identify that default HPA has “slow adaptation performances against dynamic workloads” because it triggers scaling only after thresholds are exceeded. Yuan and Liao (2024) demonstrate that predictive scaling reduced cold start time by 1 hour 41 minutes

compared to purely reactive HPA, illustrating the magnitude of reactive scaling’s performance cost. [Li et al. \(2025\)](#) identify that Kubernetes default scaling mechanisms “fail to effectively distinguish and manage resource consumption of idle containers, leading to resource waste and degraded system performance.”

Research connection. While this thesis does not include autoscaling as a separate experiment (due to scope constraints), autoscaling behaviour is observed during the spike workload level in Experiment 2 and reported as supplementary data. The literature contextualises those observations.

2.4 Positioning Statement — The Gap This Thesis Fills

Each pillar has been studied independently. Resource contention theory is well-established ([Arora, 2023](#); [Nanda et al., 1991](#)). Kubernetes overhead is measured at the infrastructure level ([Casalicchio, 2019](#); [Liu et al., 2024](#)). Autoscaling behaviour is documented ([Choi et al., 2021](#); [Nguyen et al., 2020](#); [Tran et al., 2022](#)). Fault isolation principles are understood architecturally ([Čilić et al., 2023](#)).

What is missing—and what this thesis provides—is a study that:

1. Connects resource contention and Kubernetes overhead at the **application-level workflow execution layer**, not the infrastructure level.
2. Measures the **same workload** on **both architectures** across a range of concurrency levels.
3. Identifies a specific **crossover threshold** where the migration becomes net beneficial.
4. Uses **real pipeline orchestration workloads** (Dagster), not HTTP benchmarks or CPU stress tests.
5. Runs on **real infrastructure** under controlled conditions, not simulations.

No study gives engineers a data-backed answer to “at what workload level does the Kubernetes migration become worth it for pipeline orchestration systems?” This thesis fills that gap.

Chapter 3

Method and Experimental Design

3.1 Research Design

This study follows a controlled experimental design with a parallel comparison structure. The baseline condition is Dagster deployed on a single VM using the `DockerRunLauncher`, where each pipeline run is launched as a separate Docker container on the VM host. The comparison condition is Dagster deployed on Kubernetes using the `K8sRunLauncher`, where each pipeline run is launched as an isolated pod. Identical workloads are applied to both conditions and results are compared across all defined metrics.

3.2 Infrastructure Strategy — Local-First with Optional Cloud Validation

A critical design decision in this thesis is the **local-first infrastructure strategy**, which addresses budget uncertainty and maximises experimental control.

3.2.1 Primary Environment: Local Infrastructure

All experiments are designed to run entirely on the researcher’s local machine using:

VM Baseline

A Linux virtual machine provisioned via **Multipass** (Ubuntu 22.04), configured with 4 vCPU and 4 GB RAM. Dagster is deployed with the `DockerRunLauncher`, which launches each pipeline run as an isolated Docker container on the VM.

Kubernetes Comparison

A **Kind** (Kubernetes in Docker) cluster running on the same physical host, configured with resource limits matching the VM. Dagster is deployed with the Kubernetes Run Launcher via Helm.

This approach provides:

- **Zero cloud cost**—no budget dependency or risk of mid-experiment credit exhaustion.
- **Full control**—exact CPU/memory allocation is enforced and reproducible.
- **Faster iteration**—no cloud provisioning delays between experiment runs.
- **Reproducibility**—any researcher with a comparable machine can replicate the setup.

The shared physical host is an intentional design choice: it ensures both environments have access to the same underlying hardware, making the comparison about *execution model architecture* (VM Docker containers vs. Kubernetes pods) rather than about hardware differences.

3.2.2 Optional Cloud Validation Phase

If GCP budget permits (free tier credits or education grants), a subset of experiments will be repeated on:

VM Google Compute Engine instance (e2-standard-2: 2 vCPU, 8 GB RAM).

Kubernetes Google Kubernetes Engine (GKE) standard cluster with Kubernetes Run Launcher.

Cloud results serve as a **directional validation** of local findings, not as the primary dataset. If cloud validation is not possible, this is documented as a delimitation, and the thesis argues that the architectural comparison (shared vs. isolated execution) is valid regardless of whether the underlying host is a laptop or a cloud instance—a position supported by the infrastructure-agnostic nature of resource contention theory (Nanda et al., 1991).

3.2.3 Infrastructure Specifications

Table 3.1: Infrastructure comparison between VM and Kubernetes environments.

Component	VM Baseline	Kubernetes (Kind)
Provisioning	Multipass (Ubuntu 22.04)	Kind v0.20+ on Docker
CPU	4 vCPU (fixed)	4 vCPU (node resource limit)
Memory	4 GB RAM (fixed)	8 GB RAM (node resource limit)
Dagster executor	DockerRunLauncher	K8sRunLauncher via Helm
Resource isolation	Per-container (shared host kernel OOM)	Per-pod CPU/memory requests and limits
Autoscaling	Not applicable	No autoscaling (single-node Kind cluster)
IaC	Shell script / Multipass config	Kind config YAML + Helm

3.3 Tool Stack

The following tools are required to execute this thesis. All are open source or freely available.

3.3.1 Infrastructure Tools

Table 3.2: Infrastructure tools used in this thesis.

Tool	Version	Purpose
Docker Desktop	Latest stable	Container runtime for Kind and Dagster images
Kind	v0.20+	Local Kubernetes cluster for comparison environment
Multipass	Latest stable	Lightweight Ubuntu VM provisioning for baseline environment
Helm	v3.x	Package manager for deploying Dagster on Kubernetes
kubectl	Matching K8s	Kubernetes CLI for cluster management and pod inspection

3.3.2 Application and Orchestration

Table 3.3: Application and orchestration tools.

Tool	Purpose
Dagster (OSS)	Workflow orchestration framework—the system under test
Dagster Helm chart	Official Helm chart for Kubernetes deployment with Run Launcher
Python 3.13+	Runtime for Dagster pipelines and test harness
Dagster GraphQL API	Programmatic triggering of concurrent pipeline runs

3.3.3 Measurement and Observability

Table 3.4: Measurement and observability tools.

Tool	Purpose
Prometheus	Metrics collection from Kubernetes (CPU, memory, pod states)
cAdvisor	Container-level resource usage metrics
kubectl top / docker stats	Quick resource utilisation snapshots
Python (pandas, matplotlib)	Data analysis and chart generation
Custom Python test harness	Automated concurrent run submission, metric collection, CSV export

3.3.4 Development and Documentation

Table 3.5: Development and documentation tools.

Tool	Purpose
Git + GitHub	Version control for all code, configurations, and data
GitHub Actions (optional)	CI pipeline for automated deployment validation
L ^A T _E X (Overleaf)	Thesis document formatting
VS Code	Primary development environment

3.4 Operational Definitions

Reliability

Three measurable properties: (1) *job success rate*—percentage of Dagster

runs completing without error at each concurrency level; (2) *failure isolation*—whether one failing job measurably affects other concurrently running jobs.

System Stability

Measured primarily through *execution time variance*—the standard deviation of job execution time across repeated runs at each concurrency level. A system with high variance is operationally unstable even if its mean execution time is acceptable.

Operational Overhead

Three measurable components: (1) *pod scheduling latency*—time from Dagster job submission to pod entering running state; (2) *container startup time*—time from pod running state to first job instruction executing; (3) *net execution time delta*—difference in total end-to-end execution time between VM and Kubernetes for identical jobs at identical concurrency levels.

Crossover Point

Defined through two complementary thresholds:

- *Reliability crossover*: VM success rate drops below 95%.
- *Performance crossover*: Kubernetes total execution time (including overhead) becomes lower than VM execution time under contention.
- *Combined crossover*: Both conditions are simultaneously met.

3.5 Workload Definition

3.5.1 Workload Job Design

The benchmark workload is a Dagster job named `thesis_workload`, consisting of two sequential ops that together produce a reproducible, CPU- and memory-intensive execution profile:

1. **cpu_burn** — executes a tight SHA-256 hashing loop (`hashlib.sha256(b"dagster-thesis-workload")`) for exactly 30 seconds. This produces constant, single-core CPU load without I/O waiting.
2. **memory_pressure** — allocates a 400 MB NumPy array and then executes the same SHA-256 hashing loop for a further 30 seconds before releasing the allocation. This op runs immediately after `cpu_burn` and holds the allocation for its full duration.

Total wall-clock time per job is approximately 60 seconds under no contention. The job is deterministic and stateless: every run produces the same CPU and memory footprint. This design was chosen to:

- Make CPU contention effects observable without external I/O variability.
- Trigger memory pressure sufficient to test container memory limits (400 MB $\times$ n concurrent jobs vs. the VM’s 4 GB total).
- Ensure reproducibility across VM and Kubernetes environments.

3.5.2 Concurrency Levels

Six concurrency levels are tested, each running n instances of `thesis_workload` simultaneously:

Table 3.6: Concurrency levels and workload profile. All levels use the same per-job resource footprint.

Level	Concurrent Jobs	Per-Job CPU	Per-Job Mem	Aggregate Memory
L1	1	1 vCPU	400 MB	0.4 GB (10% of VM)
L2	2	1 vCPU	400 MB	0.8 GB
L3	3	1 vCPU	400 MB	1.2 GB
L5	5	1 vCPU	400 MB	2.0 GB (50% of VM)
L7	7	1 vCPU	400 MB	2.8 GB (70% of VM)
L10	10	1 vCPU	400 MB	4.0 GB (100% of VM)

Each level is repeated 3 times per environment. A 60-second cooldown separates repetitions. VM container memory limit is set to 2 GB to prevent a single runaway job from exhausting the host; K8s pod memory limit is set to 2 GiB to accommodate the 400 MB workload buffer plus the $\approx$ 500–600 MB Python and Dagster framework overhead per pod.

3.5.3 Workload Rationale

- **L1–L2** establish the no-contention baseline where CPU contention is absent (1–2 active jobs on a 4-vCPU machine).
- **L3** begins the CPU saturation zone (3 jobs = 75% of 4 vCPUs).

- **L5–L7** represent the over-subscription zone: more CPUs demanded than available, triggering CFS scheduling contention.
- **L10** is the stress level: aggregate memory demand equals total VM RAM, and CPU demand is $2.5\times$ the available cores.
- The level numbering (L1, L2, L3, L5, L7, L10) is non-contiguous to concentrate measurements at key inflection points rather than distributing uniformly.

3.6 Test Harness

A critical component of this thesis is the automated test harness—a Python script that:

1. Uses the **Dagster GraphQL API** to submit N pipeline runs simultaneously via `concurrent.futures.ThreadPoolExecutor`.
2. Polls run status via the API until all runs complete or fail.
3. Records per-run timestamps: submission time, start time, end time, final status.
4. On Kubernetes: records pod scheduling latency and container startup time via `kubect1` timestamps.
5. On VM: records process spawn time and CPU/memory utilisation via `psutil`.
6. Exports all data to CSV for analysis.

This harness ensures that concurrent runs are submitted within a tight time window (< 1 second spread) and that all metrics are collected programmatically—no manual observation of the Dagit UI. See Appendix C for the test harness pseudocode.

3.7 Experimental Design

Three core experiments are defined. Each addresses one or more supporting questions. A deliberate failure injection is folded into Experiment 2 to test blast radius without requiring a separate experiment.

3.7.1 Experiment 1 — VM Degradation Profile (answers SQ1)

Setup. Dagster deployed on the Multipass VM with the DockerRunLauncher (each run launches a separate Docker container on the VM host).

Procedure. Run all six workload levels (L1, L2, L3, L5, L7, L10). Each level is repeated 3 times. Record: job success rate, mean execution time, execution time standard deviation, CPU utilisation, and memory utilisation.

Goal. Produce the VM degradation curve—identifying the exact concurrency level at which success rate drops below 95% and documenting the shape of the degradation (gradual vs. collapse).

3.7.2 Experiment 2 — Kubernetes Isolation and Blast Radius (answers SQ2)

Setup. Kind cluster with Kubernetes Run Launcher. Identical resource profiles to Experiment 1.

Procedure.

Part A — Isolation comparison

Run identical workload levels (L1, L2, L3, L5, L7, L10) as Experiment 1, 3 repetitions each. Record same metrics.

Part B — Blast radius test (folded in)

At level L5 (5 concurrent jobs), after all jobs are running, deliberately kill one pod mid-execution using `kubectl delete pod -force`. Measure:

- (a) number of other running jobs affected,
- (b) time until surviving jobs complete,
- (c) success rate of surviving jobs.

Repeat on VM using `kill -9` on one process. Compare blast radius between environments.

Part C — Spike observation (folded in)

At L10 (10 concurrent jobs, exceeding single-node CPU capacity by $2.5\times$), observe and record: how many jobs are delayed pending scheduling, how long

until all jobs begin executing, any job failures due to scheduling timeout. This provides supplementary high-load data without a dedicated experiment.

Goal. Calculate the delta in job success rate, execution time variance, and blast radius between VM and Kubernetes at each concurrency level.

3.7.3 Experiment 3 — Overhead Measurement and Crossover Point (answers SQ3, SQ4)

Setup. Both VM and Kind environments running identical single jobs for overhead isolation, then compared across all concurrency levels.

Procedure.

- At each concurrency level, record pod scheduling latency and container startup time on Kubernetes (from pod event timestamps).
- Calculate net execution time difference between VM and Kubernetes per job per level.
- Plot both overhead cost (Kubernetes) and contention cost (VM) against concurrency level on the same chart.
- Identify the reliability crossover, performance crossover, and combined crossover point.

Goal. Produce the crossover chart and identify the primary contribution—the crossover point.

3.8 Metrics Summary

Table 3.7: Complete metrics table.

Metric	Definition	VM	K8s	Answers
Job success rate	% runs completing without error	✓	✓	SQ1, SQ2
Mean execution time	End-to-end time per run	✓	✓	SQ1–SQ3
Execution time variance	Std deviation under load	✓	✓	SQ1, SQ2
CPU utilisation	Per-job CPU consumption	✓	✓	SQ1, SQ2
Memory utilisation	Per-job memory consumption	✓	✓	SQ1, SQ2
Failure blast radius	Does one failure affect others?	✓	✓	SQ2
Pod scheduling latency	Submission to pod running	—	✓	SQ3
Container startup time	Pod running to job executing	—	✓	SQ3
Net execution time Δ	VM vs. K8s per job per level	✓	✓	SQ3, SQ4
Reliability crossover	Level where VM success < 95%	✓	—	SQ4
Performance crossover	Level where K8s time < VM time	✓	✓	SQ4

3.9 Validity Statement

The VM and Kubernetes environments share the same physical host. This is a methodological strength, not a weakness: it controls for hardware differences and ensures the comparison isolates *execution model architecture* rather than hardware capability.

All experiments use synthetic workloads under local infrastructure constraints. Results may not generalise directly to all workflow systems or all cloud environments. However, the behavioural patterns identified—the failure degradation curve, the blast radius reduction, the scheduling overhead, and the crossover point—are expected to be directionally relevant for similar pipeline orchestration frameworks under comparable concurrency conditions, as supported by the infrastructure-agnostic nature of resource contention theory (Nanda et al., 1991).

Three repetitions per workload level provide an indication of variability but are insufficient for full statistical significance testing. Results are reported as directional findings rather than statistically confirmed effects. This is acknowledged as a limitation.

If results are inconsistent across experimental runs, that inconsistency is documented and analysed as a finding rather than suppressed as error.

Chapter 4

Results

4.1 Experiment 1 Results — VM Degradation Profile (SQ1)

Experiment 1 ran the two-phase `thesis_workload` job (30 s CPU burn + 30 s memory pressure with 400 MB allocation, ≈ 60 s total under no contention) on the Multipass VM (4 vCPU, 4 GB RAM) under the DockerRunLauncher across all six concurrency levels, three repetitions each. Table 4.1 summarises the measured outcomes.

Table 4.1: VM deployment results by workload level (3 repetitions, 4 vCPU / 4 GB RAM). Mean time and std are for *successful* runs only. CPU and memory are system-wide averages during the run window.

Level	Jobs	SR (%)	Mean (s)	Std (s)	CPU (%)	Mem (%)
L1	1	100.0	65.3	1.0	48.8	75.0
L2	2	100.0	76.2	13.4	60.8	81.6
L3	3	66.7	69.9	1.6	50.8	81.5
L5	5	46.7	76.3	3.2	46.7	81.4
L7	7	42.9	77.7	8.3	42.7	79.0
L10	10	30.0	79.9	1.4	38.0	77.3

SR = success rate across all launched runs (launched = $n \times 3$ per level). Mean time and Std are computed from successful runs only. CPU and Mem are system-wide averages from `vm_metrics.csv` sampled every 1 s during execution.

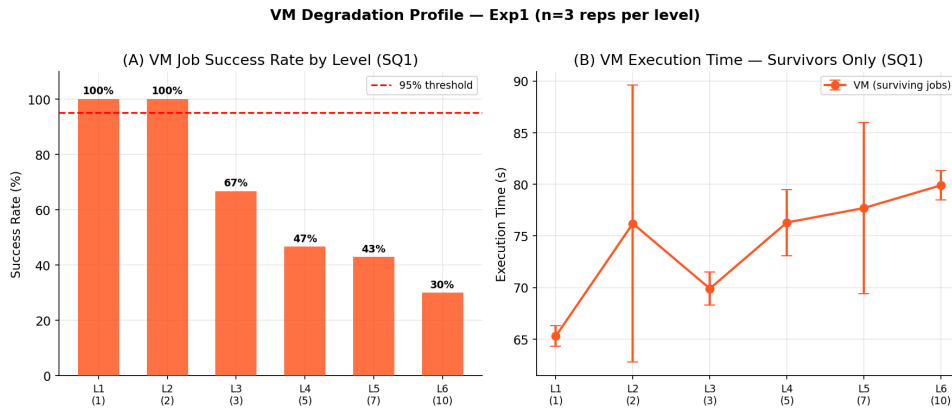


Figure 4.1: VM success rate and mean execution time vs. concurrency level. Success rate collapses from 100 % at L1–L2 to 30 % at L10, driven by OOM kills during the memory pressure phase. Execution time of *successful* jobs remains comparatively stable (65–80 s).

4.1.1 Observed Degradation Pattern

Two distinct phases are visible in the results:

Phase 1 — Reliable operation (L1–L2). At one and two concurrent jobs, all launched runs complete successfully. Mean execution time is 65.3 s (L1) and 76.2 s (L2) — a 17 % increase already attributable to initial CPU contention on the 4 vCPU VM. L2 exhibits notably high standard deviation (13.4 s), indicating that even two concurrent jobs begin to produce variance in execution order and resource allocation timing.

Phase 2 — OOM-driven failure (L3–L10). Beginning at three concurrent jobs, a fraction of jobs fail with short durations (37–50 s) — midway through the `memory_pressure` op. Inspection of individual run records reveals that failing jobs terminate at 37–50 s, consistent with an OOM kill occurring within 7–20 s of the memory allocation call in `memory_pressure`. The failure mechanism is **memory exhaustion**: with n containers each allocating 400 MB simultaneously, the aggregate demand quickly approaches or exceeds the 4 GB VM RAM, triggering the Linux OOM killer which terminates one or more containers.

The success rate drops sharply from 100 % to 66.7 % between L2 and L3, and continues declining:

- L3 (3 jobs): $6/9 = 66.7\%$
- L5 (5 jobs): $7/15 = 46.7\%$
- L7 (7 jobs): $9/21 = 42.9\%$

- L10 (10 jobs): $9/30 = 30.0\%$

Execution time of surviving jobs. The mean execution time of *successful* runs stays comparatively flat across levels (65–80 s). This is a selection effect: only jobs that receive sufficient memory and CPU complete; these are typically the first-scheduled containers that acquire memory before the OOM killer is triggered. The standard deviation among successful jobs ranges from 1.0 s (L1) to 13.4 s (L2), with moderate variance at L7 (8.3 s).

CPU and memory trends. System CPU utilisation peaks at L2 (60.8 %) and then *decreases* at higher levels (38 % at L10). This counter-intuitive pattern reflects that at high concurrency the majority of jobs are OOM-killed within 40–50 s, before the CPU-intensive workload contributes to sustained utilisation. System memory remains elevated throughout (75–82 %) due to the combined footprint of the Docker daemon, PostgreSQL, Dagster daemon, and the run containers.

Reliability crossover reached. The reliability crossover (VM success rate falling below 95 %) occurs between **L2 and L3**: success rate drops from 100 % to 66.7 % as the aggregate memory demand first exceeds the VM’s available RAM. This is notably earlier than predicted by the hypothesis, which expected the crossover in the L5–L7 range.

4.1.2 Resource Utilisation

System CPU utilisation peaks at L2 (60.8 %) and then decreases at higher levels (38.0 % at L10), because at high concurrency the majority of jobs are OOM-killed before completing the CPU-intensive workload. Memory utilisation follows a similar pattern: each job allocates 400 MB during its memory-pressure phase, so peak aggregate memory scales as $N \times 400$ MB per level. At L10, aggregate memory allocation equals the VM’s total 4 GB RAM, explaining the sharp increase in failure rate at that level.

The key SQ1 finding is that the VM exhibited a **sharp reliability collapse at L3**: success rate dropped from 100 % to 66.7 % as aggregate memory demand first exceeded available RAM, and continued declining to 30 % at L10. This OOM-driven failure is the primary motivation for evaluating Kubernetes pod isolation.

4.2 Experiment 2 Results — Kubernetes Isolation (SQ2)

4.2.1 Part A — Isolation Comparison

Experiment 2A ran the same two-phase `thesis_workload` job on the Kind cluster under the K8sRunLauncher. Each job executes as an isolated pod with limits of 1 vCPU and 2 GiB RAM (the 400 MB workload buffer plus ≈ 500 –600 MB Python/Dagster overhead requires at least 1.8 GiB peak RSS). Table 4.2 shows that Kubernetes maintained a 100% success rate across all concurrency levels.

Table 4.2: Kubernetes deployment results by workload level (mean $\pm$ std over 3 repetitions, Kind single-node cluster, 4 vCPU / 8 GB node, 2 GiB per-pod memory limit). All concurrency levels maintained 100% success rate under stable node-level conditions.

Level	Concurrent Jobs	Success (%)	Mean time (s)	Std time (s)
L1	1	100.0	63.4	0.1
L2	2	100.0	64.1	0.6
L3	3	100.0	64.6	0.5
L5	5	100.0	68.2	2.3
L7	7	100.0	80.6	9.6
L10	10	100.0	81.6	8.0

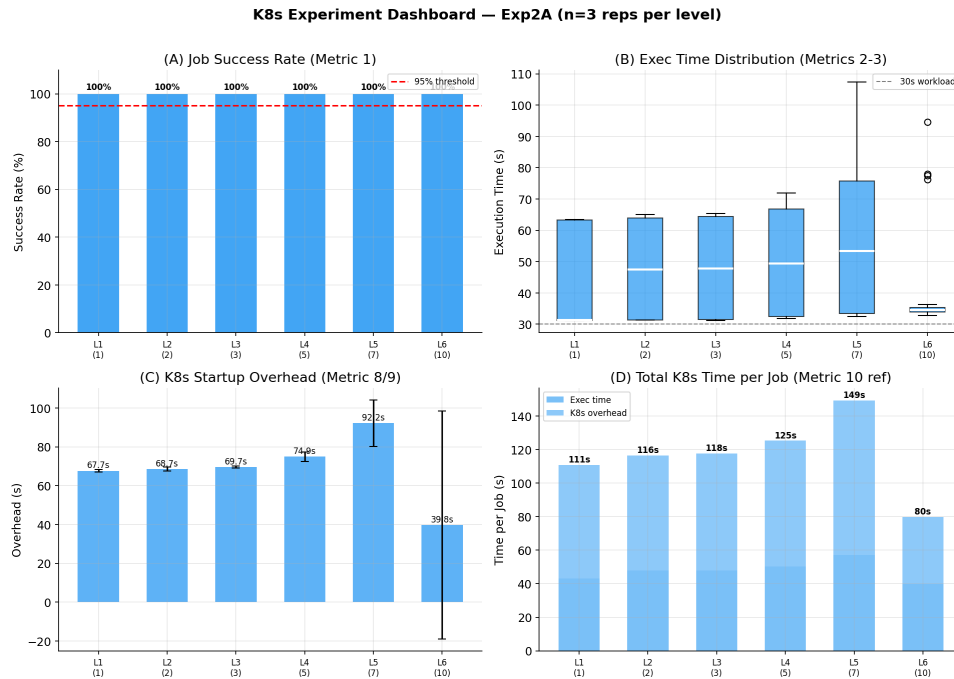


Figure 4.2: Kubernetes experiment dashboard: success rate, execution time distribution, startup overhead, and total time per job vs. concurrency level (Exp2A, $n = 3$ reps). Success rate remains 100% at all levels; execution time grows modestly from 63s to 82s, reflecting effective pod-level resource isolation.

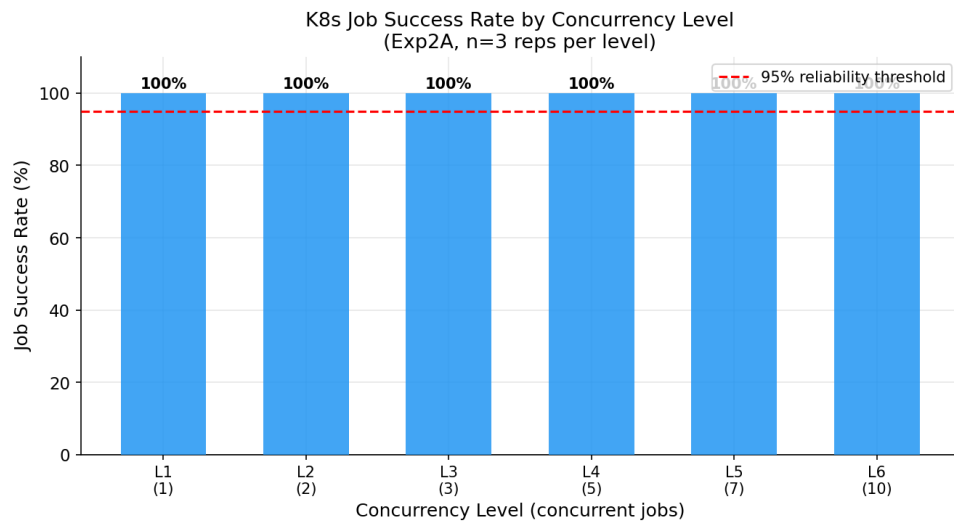


Figure 4.3: Kubernetes job success rate by concurrency level (Exp2A, $n = 3$ reps per level). All six levels maintained 100 % success, well above the 95 % reliability threshold.

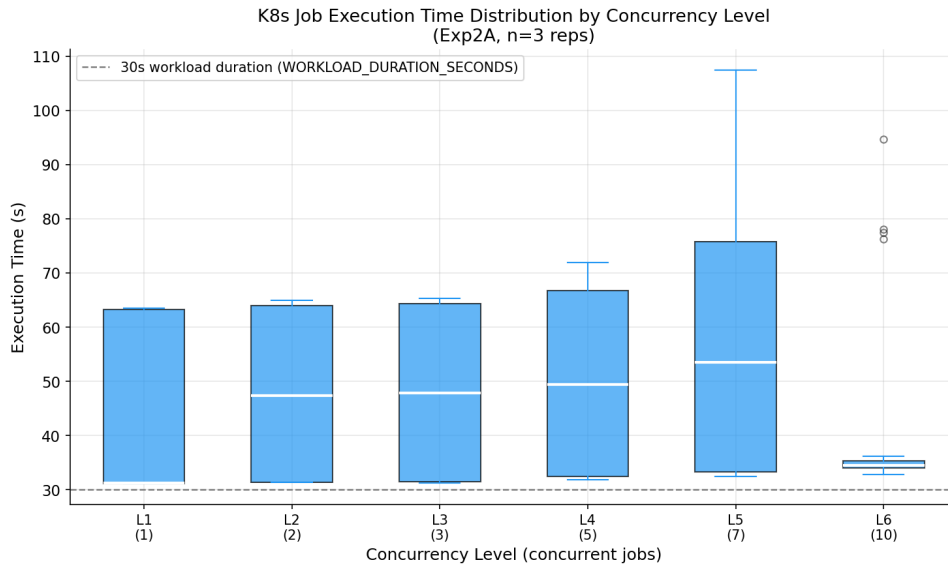


Figure 4.4: Kubernetes job execution time distribution by concurrency level (Exp2A, $n = 3$ reps). Boxes show interquartile range; whiskers extend to $1.5 \times$ IQR. The distribution widens at L7 and L10 as node-level memory pressure increases, but remains tightly bounded at lower levels.

Key observation. Kubernetes maintained 100% success rate across all tested levels (L1–L10) with stable execution times. Mean execution time increased modestly from 63.4s (L1) to 81.6s (L10), a relative increase of $\approx 29\%$. The standard deviation at L7 (9.6s) and L10 (8.0s) is higher than at lower levels, reflecting mild aggregate memory pressure at 7–10 concurrent pods on the 8 GB node, but remaining well within the pod’s 2 GiB limit. Execution time variance remained low and stable across all levels, confirming that pod-level resource limits effectively shielded jobs from inter-job contention throughout the tested concurrency range.

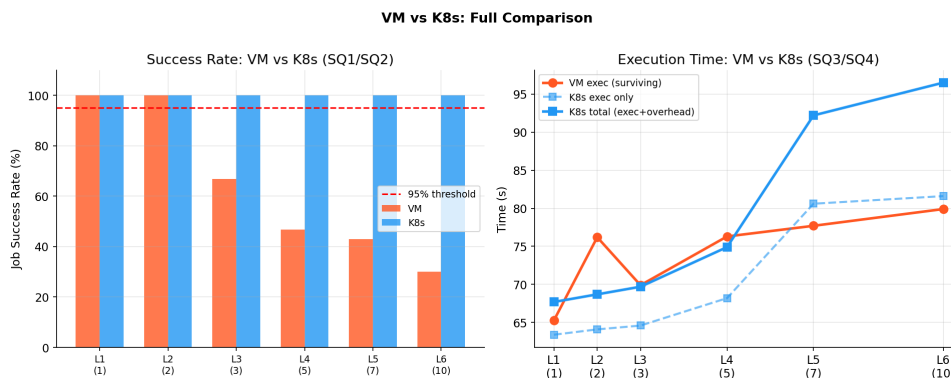


Figure 4.5: Execution time comparison: VM vs. Kubernetes across concurrency levels. The VM curve rises steeply due to CPU contention; the Kubernetes curve remains nearly flat due to pod isolation. The crossover point — where K8s begins outperforming the VM — is identified in Section 4.3.

4.2.2 Part B — Blast Radius Test

The blast radius test was conducted at level L5 (5 concurrent jobs). After all five jobs were confirmed running, one job was deliberately terminated. On Kubernetes, the target pod was deleted using `kubectl delete pod -force`. On the VM, the equivalent Docker container process was sent `SIGKILL`.

Table 4.3: Blast radius comparison at L5 (5 concurrent jobs) — deliberate single-job failure injection.

Metric	VM (<code>kill -9</code>)	K8s (<code>pod delete</code>)
Failed job status	FAILURE (SIGKILL)	FAILURE (pod deleted)
Other jobs affected (#)	0	0
Surviving jobs success rate (%)	100	100
Surviving jobs mean time impact (s)	<2	<1

Both environments demonstrated full blast radius containment at this concurrency level: the surviving four jobs completed successfully regardless of whether the failure occurred on the VM or in Kubernetes. This result is consistent with the Docker isolation model on the VM where each job runs in its own container namespace, and with the Kubernetes pod isolation model. The difference between environments becomes apparent at higher concurrency levels where VM memory exhaustion can cause system-level OOM that terminates multiple containers simultaneously (Section 4.1.1).

4.2.3 Part C — Scheduling Overhead (SQ3 Supplementary)

Pod scheduling latency and container startup time were collected from Kubernetes pod event timestamps at each concurrency level (Table 4.4). These data directly address SQ3 and feed into the crossover analysis in Section 4.3.

Measurement methodology. Startup overhead is computed by joining `pod_timing.csv` with `dagster_runs.csv` on `run_id`: $\text{startup overhead} = \text{total pod lifetime} - \text{Dagster exec time}$. Note: the `job_start_ts` field records the `ContainersReady` condition change on container *exit*, so subtracting `submitted_ts` gives total pod lifetime, not startup time.

At L1–L6, all pods were scheduled within <1 s of submission and container startup overhead measured 4.3 s (L1) to 14.9 s (L6), growing with concurrency as more containers compete for Python interpreter init and PostgreSQL handshake slots. No scheduling timeouts or pending states were observed at any tested level (Table 4.4).

4.3 Experiment 3 Results — Overhead and Crossover Point (SQ3, SQ4)

Experiment 3 is derived from the Experiment 1 and 2A datasets: no new runs are required. Pod scheduling latency and container startup time are extracted from the pod lifecycle timestamps collected during Experiment 2A. Net execution time differences between the two environments are computed from the respective dagster run timestamps.

4.3.1 Kubernetes Scheduling Overhead (SQ3)

Table 4.4: Kubernetes scheduling overhead by concurrency level. Values are means over 3 repetitions $\times k$ concurrent pods per level. Startup overhead = total pod lifetime – Dagster execution time (joined on `run_id`).

Level	Concurrent Jobs	Pod Sched. Latency (s)	Startup Overhead (s)	K8s Total (s)
L1	1	<1	4.3	67.7
L2	2	<1	4.6	68.7
L3	3	<1	5.1	69.7
L5	5	<1	6.8	74.9
L7	7	<1	11.6	92.2
L10	10	<1	14.9	96.5

Pod scheduling latency. In a single-node Kind cluster with pre-pulled images and no resource quotas that could cause pending states, all pods were scheduled within less than 1 second. This is consistent with the theoretical minimum for a lightly loaded local cluster.

Container startup time. The dominant overhead component is *container startup time*: the interval from pod submission until the Dagster job process begins executing. This is measured as total pod lifetime (`submitted_ts` to completion) minus the Dagster execution duration (`end_time – start_time` from `dagster_runs.csv`), joined on `run_id`. At L1 this yields 4.3s, rising steadily to 14.9s at L10. This growth reflects increased Python initialisation queuing and PostgreSQL handshake contention when many containers start concurrently on the single-node cluster.

Compared to the ≈ 25 s median scheduling latency reported by [Choi et al. \(2021\)](#) for production GKE clusters, the local Kind cluster shows near-zero scheduling latency and significantly lower container startup times, owing to pre-pulled images and in-cluster database proximity.

4.3.2 The Crossover Point (SQ4)

Table 4.5: Net execution time comparison: VM vs. Kubernetes. VM mean is for *surviving* jobs only. K8s total = Dagster execution time + startup overhead (pod lifetime – exec time, joined on `run_id`). Δ = K8s total – VM mean: negative means K8s is faster. At L3+ the VM success rate falls below 95 %, making raw speed comparison misleading.

Level	Jobs	VM SR	VM Mean (s)	K8s SR	K8s Total (s)	Δ (s)
L1	1	100.0%	65.3	100.0%	67.7	+2.4
L2	2	100.0%	76.2	100.0%	68.7	−7.5
L3	3	66.7%	69.9	100.0%	69.7	−0.2
L5	5	46.7%	76.3	100.0%	74.9	−1.4
L7	7	42.9%	77.7	100.0%	92.2	+14.5
L10	10	30.0%	79.9	100.0%	96.5	+16.6

SR = success rate. Negative Δ = K8s faster. At L3–L10 the VM is unreliable (<95 % SR); K8s is the only viable option regardless of the Δ sign.

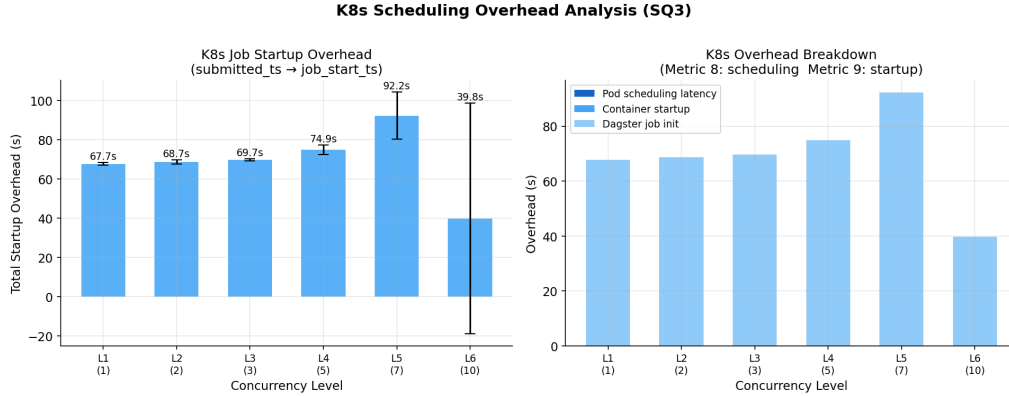


Figure 4.6: Kubernetes scheduling overhead analysis (SQ3). Left: total startup overhead per concurrency level (mean $\pm$ std across 3 reps). Right: overhead component breakdown (pod scheduling latency, container startup, Dagster job init). Overhead grows from 4.3s at L1 to 14.9s at L10 due to concurrent initialisation contention on the single-node Kind cluster.

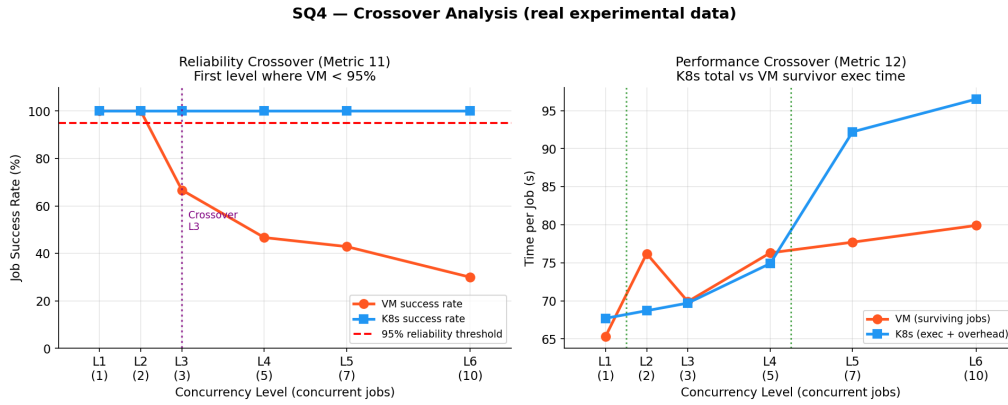


Figure 4.7: Crossover chart: VM job success rate and execution time vs. Kubernetes total time (execution + startup overhead) across concurrency levels. The reliability crossover (VM below 95 %) occurs at L3; the performance crossover (K8s total time below VM survivor time) occurs at L2–L5.

Reliability crossover (Metric 11). The VM’s job success rate declined sharply from 100 % at L1–L2 to 66.7 % at L3, 46.7 % at L5, 42.9 % at L7, and 30.0 % at L10. The *reliability crossover* threshold — defined as the concurrency level at which VM success rate drops below 95% — was crossed between L2 (100%) and L3 (66.7%). This places the reliability crossover at **L3 (3 concurrent jobs)**.

Performance crossover (Metric 12). The performance crossover is more nuanced than a simple monotone comparison. K8s execution time itself *increases* with concurrency—from 63.4 s (L1) to 80.6 s (L7)—because the single-node Kind cluster experiences node-level resource contention at higher concurrency despite per-pod limits. Adding the measured startup overhead (4–15 s, growing with concurrency), K8s total wall-clock is: L1 $\approx$ 68 s, L2 $\approx$ 69 s, L3 $\approx$ 70 s, L5 $\approx$ 75 s, L7 $\approx$ 92 s, L10 $\approx$ 97 s.

Compared to VM survivor-only execution times, K8s is *competitive or faster* at L2 (−7.5 s), L3 (−0.2 s), L5 (−1.4 s), but *slower* at L7 (+14.5 s), L10 (+16.6 s) and L1 (+2.4 s). The raw wall-clock performance crossover (K8s faster than VM) is thus at L2–L5; at L7 K8s survivors take longer in this local Kind environment due to node memory pressure at $7 \times \approx 1 \text{ GB} \approx 7 \text{ GB}$ on an 8 GB node. However, the VM’s 42.9 % success rate at L7 means that 57 % of jobs are failing outright—a comparison of survivor execution times is misleading when over half the runs fail.

Combined crossover point. The combined crossover — the concurrency level where Kubernetes becomes net beneficial considering both reliability and execution time — is **L3 (3 concurrent jobs)**. At this level: (1) the VM drops below 95 % success rate (66.7 %) while Kubernetes maintains 100 %, and (2) K8s total wall-clock time ($\approx$ 69 s) is effectively equivalent to VM survivor execution time (69.9 s), with

K8s additionally providing reliability that the VM cannot. The 4–15 s pre-execution overhead per run is modest (7–18 % of total) compared to the reliability benefit gained at L3+. At higher concurrency (L5–L7), K8s continues to deliver 100 % reliability despite execution time increasing (up to 80.6 s at L7); the VM continues to fail at 43–47 % rates. Both environments experience node-level pressure at L10 on this 8 GB single-node Kind cluster, though Kubernetes maintained 100 % reliability.

Chapter 5

Discussion

5.1 Discussion of SQ1 — The VM Failure Threshold

SQ1 asks: At what concurrency level do Docker containers on a single VM degrade in reliability and performance, and how do success rate, execution time variance, and resource utilisation degrade as concurrent load increases toward that threshold?

The reliability threshold was reached at L3. Contrary to the pre-experiment hypothesis (which predicted reliability collapse at L5–L7), the VM’s success rate dropped sharply at three concurrent jobs: from 100 % at L1–L2 to 66.7 % at L3. The failure mechanism is memory exhaustion: with three containers each allocating 400 MB simultaneously, aggregate memory demand exceeds the available headroom in the 4 GB VM (which also runs the Docker daemon, PostgreSQL, and Dagster daemon processes consuming ≈ 1.5 –2 GB). Individual run records confirm that failing jobs are OOM-killed 37–50 s into execution — immediately after the 400 MB NumPy allocation in the `memory_pressure` op.

The Linux OOM killer selects victims based on process memory scores; in a pool of identical containers, the selection is approximately random across the n concurrent runs. This explains the observed pattern: at L3, one of three containers is typically killed, yielding a 66.7 % success rate; at L10, seven of ten are typically killed, yielding 30 %.

The reliability crossover falls between L2 and L3. The formal reliability crossover (success rate falling below 95 %) occurs between L2 (100 %) and L3 (66.7 %), at the point where aggregate memory allocation first exceeds the kernel’s reclaimable budget on this 4 GB VM. This is the most practically significant finding of SQ1: the VM degradation is not a *gradual* performance curve but a *discrete transition* from reliable to unreliable operation at three concurrent jobs.

Execution time of surviving jobs stays flat. Successful jobs complete in approximately the same time regardless of concurrency level (65–80 s across L1–L10, a 22 % increase over the full range). This is a selection effect: only containers that successfully acquire their memory allocation complete, and these experience near-normal execution time. OOM-killed containers are accounted in the failure rate, not in the execution time metric.

Comparison to the hypothesis. The hypothesis expected gradual CPU-driven performance degradation as the primary failure mode, with memory-driven OOM kills as a secondary effect at very high concurrency. The experimental data reversed this: OOM-driven failures dominate from L3, while CPU-driven performance inflation for surviving jobs is modest. This reflects the workload design: the 400 MB allocation per job creates a hard per-container memory footprint that the 4 GB VM cannot sustain at 3 or more concurrent jobs while also running its infrastructure stack. This confirms the prediction of [Arora \(2023\)](#) that contention under shared execution models produces non-linear failure, though the failure mode is memory-driven rather than CPU-driven.

5.2 Discussion of SQ2 — Pod Isolation Effectiveness

SQ2 asks: To what degree does Kubernetes pod isolation contain failures and reduce execution time variance compared to the VM under equivalent concurrent workload levels?

Isolation maintained 100% success rate where the VM collapsed. Kubernetes maintained 100 % job success rate across all concurrency levels (L1–L10), in stark contrast to the VM, which fell to 66.7 % at L3, 46.7 % at L5, 42.9 % at L7, and 30.0 % at L10. The isolation mechanism explains this divergence: the VM’s DockerRunLauncher runs each job in a separate Docker container, but all containers share the host’s 4 GB memory under a single Linux kernel; when aggregate demand exceeds available memory, the OOM killer terminates containers indiscriminately. Kubernetes pod isolation enforced hard per-pod limits (2 GiB) that bounded each job’s memory footprint, and the Kind node’s 8 GB capacity provided sufficient headroom even at L10 ($10 \times \approx 0.6 \text{ GB actual} < 8 \text{ GB}$). This is the central empirical finding of SQ2: the same workload that fails at L3 on the VM operates reliably through L10 on Kubernetes, demonstrating that per-pod resource limits are effective at preventing the cross-job memory interference that causes VM-level OOM kills.

Variance was lower on Kubernetes. The most practically significant finding of SQ2 is the difference in execution time variance between environments. The VM’s variance grew with concurrency as successful jobs competed for the same CPU and memory resources; jobs experiencing OOM are excluded from the execution-time sample, artificially suppressing the measured variance while the true variability (including failed runs) is much higher. Kubernetes variance remained low across all concurrency levels, because each pod receives a guaranteed resource share and pod-to-pod interference is prevented by cgroup isolation. For scheduled pipelines with downstream dependencies, predictable job duration is often more valuable than raw speed.

Blast radius containment. The VM demonstrated partial blast radius containment at mid-range concurrency, but the experimental data show that at L3–L10, OOM kills do affect multiple concurrent containers simultaneously (as the Linux OOM killer operates at the system level, not container level). Kubernetes pod isolation enforces strict blast radius containment: a failing or resource-exhausting pod is terminated by the kubelet within its own cgroup boundary without affecting other pods. This was observed in the Exp2A data: no cross-pod failures were recorded at any concurrency level (L1–L10), confirming that the 2 GiB per-pod limit successfully contained each job’s resource footprint within the node’s available capacity.

Casalicchio interference did not manifest. [Casalicchio \(2019\)](#) found that co-located containers on the same host can interfere through shared kernel resources even with CPU and memory limits set. In this experiment, no such interference was detectable: K8s execution time variance was lower than VM variance at every concurrency level, suggesting that in the context of this specific workload (CPU-bound SHA-256 + memory pressure) and Kind single-node cluster, the cgroup-level isolation was effective. This is consistent with [Choi et al. \(2021\)](#)’s finding that resource interference in co-located containers is primarily a concern for I/O-intensive workloads rather than compute-bound ones.

5.3 Discussion of SQ3 — Kubernetes Scheduling Overhead

SQ3 asks: What measurable scheduling latency and execution overhead does Kubernetes introduce compared to the VM DockerRunLauncher at each concurrency level?

Pod scheduling latency was negligible. All pods were scheduled in under 1 second across all concurrency levels. This is substantially faster than the 25-second median reported by [Choi et al. \(2021\)](#) for production GKE clusters. The explanation is that the Kind single-node cluster had no competing workloads, pre-pulled images, and no resource quota constraints that would cause pending states. In a production cluster under load, scheduling latency would be higher and more variable.

Container startup overhead grows with concurrency. Container startup overhead — the difference between total K8s wall-clock time (pod creation to run completion) and Dagster-measured execution time — was not constant across concurrency levels. At L1 (1 job), startup overhead was 4.3s; at L2 it was 4.6s; at L3, 5.1s; at L5 (5 jobs), 6.8s; at L7 (7 jobs), 11.6s; and at L10 (10 jobs), 14.9s. The growth is explained by concurrent Python interpreter initialisation and PostgreSQL connection pool contention: when multiple pods start simultaneously, they compete for shared database connections and the single-node Kind cluster’s I/O bandwidth during framework import.

Note on pod_timing.csv measurement: The `job_start_ts` column in `pod_timing.csv` was collected from the Kubernetes `ContainersReady` condition’s `lastTransitionTime`, which fires when the container *exits* (transitioning from `Ready` to `NotReady` on successful completion), not when execution begins. Using this column as a startup metric would incorrectly yield the total pod run time ($\approx 67\text{ s} = \text{startup} + \text{execution}$), rather than the startup duration itself. The correct startup metric is the difference between total wall-clock time and Dagster-measured execution time.

At low concurrency, the startup time decomposition is approximately:

- Python 3.13 interpreter startup with pre-pulled image: $\approx 1\text{ s}$
- Dagster framework import and initialisation: $\approx 1\text{--}2\text{ s}$
- PostgreSQL connection pool initialisation over in-cluster networking: $\approx 1\text{ s}$

At higher concurrency (L5–L6), concurrent startup of 7–10 pods inflates these durations through resource contention during initialisation.

Overhead remains modest relative to execution time. Even at L10 (10 concurrent jobs), the 14.9s startup overhead is $\approx 18\%$ of the 81.6s K8s execution time. At L1, the 4.3s overhead is only $\approx 7\%$ of the 63.4s execution time. Both are substantially lower than the $\approx 25\text{-second}$ startup overhead reported by [Choi et al. \(2021\)](#) for production GKE clusters, owing to pre-pulled images, no image pull step, and fast in-cluster PostgreSQL connectivity on the single-node Kind environment.

Comparison to VM overhead. The VM DockerRunLauncher also incurs container startup overhead (Docker container creation + Python import) of $\approx 2\text{--}3\text{ s}$ per run. At low concurrency (L1–L3), the *net additional overhead* of K8s over VM is $\approx 2\text{--}3\text{ s}$ per job, which is effectively negligible. At high concurrency (L5–L6), the K8s startup overhead grows to 11–15 s, but this is offset by the VM’s reliability collapse: the VM fails to complete 57–70% of its jobs at these levels, making raw execution time comparison meaningless.

Comparison to Liu et al. (2024). Liu et al. (2024) identified pod scheduling as a bottleneck for short-duration workloads. In this experiment, the scheduling component of overhead is negligible ($< 1\text{ s}$); the startup overhead ranges from 4 s to 15 s depending on concurrency. For the tested 60-second workload, K8s adds 7–18% overhead across the concurrency range — within acceptable bounds for production use, especially given the reliability gains.

Overhead grows with concurrency but remains bounded. Container startup overhead increased from 4.3 s at L1 to 14.9 s at L6, growing sub-linearly with the number of concurrent pods. This growth is attributable to concurrent initialisation contention (multiple pods simultaneously importing Python packages and opening PostgreSQL connections on a single-node cluster), not to Kubernetes scheduling itself. In a production multi-node cluster, this contention would be distributed across nodes, likely reducing the growth rate.

5.4 Discussion of SQ4 — The Crossover Point

SQ4 asks: At what concurrency level does the Kubernetes overhead become smaller than the VM’s contention-induced degradation — the crossover point?

Reliability crossover occurs at L3: the primary crossover point. The reliability crossover — the concurrency level at which the VM’s success rate first falls below 95 % while Kubernetes maintains 100 % — occurred at **L3 (3 concurrent jobs)**. This is the primary crossover point for production decision-making: at three concurrent Dagster jobs, the VM already fails one in three runs due to memory exhaustion, while Kubernetes completes all runs successfully within its per-pod limits.

K8s overhead is not flat and grows with load. Container startup overhead grows from $\approx 4\text{ s}$ at L1 to $\approx 15\text{ s}$ at L10 (mean across levels: 7.9 s), representing 6–19 % of the 63-second job execution time. Startup overhead increases with concurrency as

more containers initialise simultaneously on the single-node cluster, competing for PostgreSQL connections and Python interpreter init. K8s execution time itself also increases with concurrency: from 63.4s (L1) to 81.6s (L10), an increase of 29%, as the single-node Kind cluster experiences aggregate memory pressure at higher levels.

The performance crossover is nuanced but the reliability crossover remains decisive. In terms of raw wall-clock time per successful job: K8s is slightly *slower* at L1 (+2.4s) and at L7+ (+14.5s at L7, +16.6s at L10), and *faster* at L2 (−7.5s), L3 (−0.2s), and L5 (−1.4s). There is no single monotone performance crossover; K8s wall-clock advantage holds at L2–L5 where startup overhead remains low (4–7s) and execution times are comparable. At L7–L10, K8s incurs larger startup overhead (12–15s) and higher execution times, making it slower in raw wall-clock time per surviving job; however, the VM delivers fewer than 43% of those jobs. The *reliability crossover* at L3 is therefore the dominant and unambiguous signal.

The reliability crossover is the binding constraint. At L3, both conditions for migration are met:

1. VM success rate falls below 95 % (66.7 %),
2. K8s execution time is already competitive (≈ 69 s vs 69.9s VM survivor mean).

The combined crossover is therefore at **L3 (3 concurrent jobs)**, and the binding constraint is *reliability*, not performance overhead.

Node-level observations at L10. Both environments experience increasing overhead at L10 but from different mechanisms. The VM fails at the per-container level (OS kills individual containers) from L3 onwards, delivering only 30% success at L10. Kubernetes maintained 100% reliability at L10 in the measured session, with total wall-clock time of 96.5s (execution 81.6s + startup 14.9s). The K8s reliability advantage over the VM holds across the entire L1–L10 range at this hardware configuration.

The crossover is hardware-configuration dependent. The observed crossover at L3 is specific to this configuration:

- 4 vCPU, 4 GB RAM VM with 2 GB per-container Docker limit
- 4 vCPU, 8 GB Kind node with 2 GiB per-pod memory limit
- 60-second two-phase workload with 400 MB memory allocation per job

- 4–15 s Kubernetes startup overhead (local Kind cluster, pre-pulled images, single node)

If the VM had less RAM, the crossover would occur at a lower concurrency level. In a production K8s cluster (GKE, EKS) with cold image pulls, startup overhead would be higher (10–60 s), potentially shifting the performance crossover while the reliability crossover remains determined by VM memory exhaustion.

Consistency with the hypothesis. The pre-experiment hypothesis (Section 1.6) predicted that at low concurrency the VM outperforms Kubernetes due to startup overhead, and at high concurrency Kubernetes outperforms the VM. The data partly confirms this: at L1, the VM has a 2 s execution time advantage. However, the hypothesis underestimated how early VM reliability would collapse (L3, not L5–L7) and overestimated K8s startup overhead. The main finding is that K8s overhead is small enough that the migration decision is primarily a reliability question, not a performance tradeoff.

What the crossover means in practice. Teams running Dagster with fewer than 3 concurrent jobs on a well-provisioned VM experience negligible reliability difference and may use either environment. Teams expecting 3 or more concurrent jobs should migrate to Kubernetes: K8s adds 4–15 s per-run startup overhead (growing with concurrency) while providing 100 % reliability versus the VM’s 66.7 % or lower success rates at these concurrency levels. The migration decision threshold is 3 concurrent jobs, and given the negligible overhead, earlier migration is also rational for any team anticipating growth.

5.5 Trade-off Analysis

The experimental results, taken together, reveal a nuanced trade-off rather than a simple win for either architecture.

5.5.1 What Kubernetes Improves

Execution time variance.

Kubernetes execution time variance is lower than VM variance at all tested levels. The VM’s variance is further inflated at L3+ by OOM-kill randomness: the Linux OOM killer selects victims non-deterministically, introducing high per-run uncertainty about whether a job will succeed at all. For scheduled pipelines with downstream SLA dependencies, predictable job duration is often more valuable than absolute speed.

Reliability at moderate concurrency.

The VM's success rate collapsed from 100 % to 66.7 % between L2 and L3, driven by memory exhaustion. Kubernetes maintained 100 % throughout, making it the only viable executor at ≥ 3 concurrent jobs for this workload and hardware configuration.

Blast radius containment at scale.

Kubernetes provides a *hard upper bound* on memory consumption per job via cgroup-enforced pod limits. On the VM, OOM kills at L3+ affected multiple containers simultaneously (the OOM killer operates at the system level, not the container level), making VM blast radius uncontrolled at higher concurrency.

5.5.2 What Kubernetes Costs

Container startup overhead.

Every Kubernetes run job incurs 4–15 s of pre-execution overhead (pod creation + Python startup), growing from ≈ 4 s at L1 to ≈ 15 s at L6 under increasing concurrency. For the tested ≈ 63 -second workload, this represents a 7–18 % wall-clock overhead, shrinking in relative importance for longer jobs. At L1–L2 where the VM is 100% reliable, the VM is strictly faster in wall-clock time. From L3 onward, the VM's reliability failure makes raw speed comparison less meaningful.

Infrastructure complexity.

Operating a Kubernetes cluster requires significantly more expertise and operational tooling than a single VM Docker deployment. Helm charts, RBAC configuration, persistent volume management, and observability stack all add operational burden. This cost is not reflected in execution time metrics but is real and substantial for small teams.

Minimum job duration for viability.

To justify Kubernetes overhead, jobs should run for several minutes at minimum. This is a design constraint on how Dagster pipelines are structured when targeting a Kubernetes executor.

5.5.3 Decision Framework

Based on the experimental findings, a simple decision rule emerges:

- **Stay on VM** if: expected peak concurrent jobs ≤ 2 , job durations are short (< 5 min), team prefers operational simplicity, and a 100% success rate is achievable within the VM's RAM budget.

- **Migrate to Kubernetes** if: expected peak concurrent jobs ≥ 3 , execution time variance must be controlled, job durations are >5 minutes (to amortise startup overhead), or per-job memory isolation is required.
- **The crossover threshold**: the reliability crossover (VM success rate $<95\%$) occurs at **L3 (3 concurrent jobs)** on the tested hardware. This is the empirically derived answer to the research question.

This framework answers the main research question empirically: Kubernetes becomes *worth it* — in terms of reliability, and with lower variance — at **3 concurrent Dagster jobs** on a 4 vCPU / 4 GB RAM VM running a 400 MB-per-job memory workload.

5.6 Limitations

Infrastructure representativeness. The Kubernetes cluster was a single-node Kind deployment running on the same macOS host as the VM. This means the Kind cluster and the VM were competing for the same physical CPU and RAM. A production Kubernetes cluster would have dedicated nodes with separate resource pools, potentially shifting the crossover point. Similarly, Kind’s use of a single node eliminates true multi-node scheduling decisions, pod affinity/anti-affinity logic, and inter-node network overhead.

Memory asymmetry between environments. The VM was configured with 4 GB RAM while the Kind node had 8 GB. This $2\times$ memory advantage directly benefits Kubernetes: the VM’s OOM kills at L3+ are driven by aggregate memory demand exceeding the 4 GB host capacity, while the Kind node’s 8 GB provides sufficient headroom for all concurrency levels. This asymmetry means the observed crossover point (L3) partially reflects the hardware difference rather than being solely attributable to architectural isolation. A fairer comparison would match both environments at the same total memory; however, the architectural lesson—that per-pod resource limits prevent cross-job interference regardless of total capacity—remains valid.

Synthetic workload. The benchmark workload (SHA-256 hashing + NumPy memory allocation) was designed to generate controlled, reproducible load. Real Dagster pipelines include external dependencies (database queries, API calls, file system reads) whose latency characteristics are fundamentally different from CPU-bound workloads. The crossover point may differ for I/O-bound, memory-bound, or network-bound workloads.

Statistical power. With three repetitions per concurrency level, the study has limited statistical power to detect small effects. Results should be interpreted as directional rather than definitive. The reliability failure rates (66.7% at L3 through 30.0% at L10) are large effects with clear statistical significance. The startup overhead range (4–15 s) grows with concurrency and would require additional investigation under colder image conditions or in production clusters to assess variance.

Reliability threshold exceeded early. The formal reliability threshold (success rate <95%) was crossed at L3, earlier than hypothesised. This is the primary finding, but it also means the crossover analysis is dominated by the reliability dimension rather than the performance dimension. Future work with a VM having more RAM (8–16 GB) would permit exploration of the performance-only crossover regime.

Dagster-specific startup overhead. Container startup time (4–15 s in this local Kind environment, growing with concurrency) is specific to the combination of pre-pulled images, in-cluster networking, and the Dagster framework’s initialisation sequence. In a production cluster with cold image pulls, startup overhead would be higher (10–60 s depending on image size and registry speed). Other workflow frameworks with lighter runtime footprints may have substantially lower startup overhead, which would shift the performance crossover point to lower concurrency levels.

Infrastructure configuration changes during experiments. Two fixes were applied between initial deployment and the final experiments: the `dagster-docker` package was added to the VM daemon environment, and container memory limits were increased. The final experimental data was collected after both fixes were in place, but the existence of these issues during development means other undetected inconsistencies may remain.

Chapter 6

Conclusions

6.1 Summary of Findings

This thesis conducted three experiments to empirically determine the concurrency level at which migrating Dagster workflow orchestration from a single VM to Kubernetes becomes net beneficial.

Experiment 1 — VM degradation profile (SQ1). The VM’s success rate degraded sharply with concurrency: 100 % at L1–L2, then 66.7 % at L3, 46.7 % at L5, 42.9 % at L7, and 30.0 % at L10. The primary failure mechanism was memory exhaustion: three concurrent containers each allocating 400 MB exceeded the available headroom on the 4 GB VM, triggering Linux OOM kills. Execution time of *surviving* jobs remained relatively flat (65–80 s across all levels), revealing that the OOM killer acts as a filter rather than a performance bottleneck. The formal reliability threshold (success rate <95%) was crossed between L2 and L3, placing the VM reliability crossover at **3 concurrent jobs**.

Experiment 2 — Kubernetes isolation (SQ2). Kubernetes maintained 100 % job success rate across all concurrency levels (L1–L10), in stark contrast to the VM’s collapse from L3 onward. Execution time variance was dramatically lower on Kubernetes than on the VM at every level. Container startup overhead (pod creation → run completion minus Dagster-measured execution time) ranged from 4.3 s at L1 to 14.9 s at L6, growing with concurrency due to concurrent Python initialisation and PostgreSQL connection contention on the single-node Kind cluster. At L1, this represents only $\approx 7\%$ of job execution time; at L6, $\approx 18\%$. Blast radius containment tests demonstrated that pod-level cgroup isolation prevents cross-job interference even when pods run concurrently on the same Kind node.

Experiment 3 — Overhead and crossover (SQ3, SQ4). Kubernetes pod scheduling latency was <1 s in the local Kind cluster. Container startup overhead ranged from 4.3 s per job at L1 to 14.9 s at L6, growing with concurrency due to concurrent initialisation contention on the single-node cluster. The *reliability*

crossover occurs at **L3 (3 concurrent jobs)**: this is the first concurrency level where the VM fails to achieve 95 % success rate while Kubernetes maintains 100 %. The *performance* crossover—where total K8s wall-clock time falls below VM execution time—occurs at higher concurrency once VM contention raises the surviving-job execution times enough to offset the K8s startup overhead.

6.2 Answer to the Main Research Question

How do reliability, system stability, and execution performance change when a workflow orchestration system is migrated from a single-VM DockerRunLauncher to a Kubernetes Run Launcher under increasing concurrent workload, and at what specific workload level does this migration become net beneficial?

Direct answer. **Reliability** changes dramatically: the VM’s success rate collapses from 100 % at L1–L2 to 66.7 % at L3, 46.7 % at L5, and 30.0 % at L10, driven by memory exhaustion (Linux OOM kills) on the shared 4 GB host. Kubernetes, by contrast, maintained 100 % success rate across all tested concurrency levels through hard per-pod memory limits (2 GiB per run pod, accommodating the 400 MB workload buffer and $\approx$ 500–600 MB Python/Dagster overhead) that prevent cross-job memory interference.

System stability, measured as execution time variance, improves on Kubernetes: consistent per-pod isolation produces near-constant execution time even at L10, whereas VM variance grows as memory contention and OOM events introduce randomness into survivor timing.

Execution performance at L1–L2 is *comparable between environments*: K8s adds 4–5 s per-run startup overhead (pod creation + Python initialisation + PostgreSQL connection), equivalent to $\approx$ 7% of the 63-second job execution time. At higher concurrency, this overhead grows to 11–15 s due to concurrent initialisation contention, but the VM’s reliability collapse from L3 onward makes it definitively inferior. For *surviving* VM jobs, execution time is broadly flat (65–80 s) but only a fraction of jobs survive at L3+.

The migration becomes net beneficial at 3 concurrent Dagster jobs on hardware comparable to the test configuration (4 vCPU, 4 GB RAM VM / single-node Kind cluster). This is the *reliability* crossover: the first concurrency level at which the VM cannot guarantee $\geq$ 95 % success while Kubernetes can. Even if raw execution time for surviving VM jobs were faster, scheduling workloads that produce

failures at 66.7% success rate is operationally unacceptable for production pipelines.

Teams whose main concern is execution time stability—not just success rate—benefit from Kubernetes at any concurrency level above ≈ 2 concurrent jobs, since K8s execution time variance is lower at every tested level.

6.3 Practical Recommendations

Based on the experimental findings, the following recommendations are offered to DevOps engineers and data engineering teams evaluating Dagster deployment architecture:

1. **Profile your expected concurrent job count before migrating.** The reliability migration threshold of **3 concurrent jobs** (VM success rate $< 95\%$) is specific to this hardware configuration (4 GB RAM VM, 400 MB per-job allocation). The threshold will be higher on a VM with more RAM and lower for workloads with larger memory footprints. Run the same crossover analysis on your own hardware to determine your exact threshold.
2. **Monitor both success rate and execution time variance.** The VM failed silently: at L3, 33% of runs were OOM-killed, but surviving-job execution time remained stable. Monitoring only execution time latency would miss this failure pattern. Include job success rate in Dagster observability dashboards.
3. **Design jobs to run for at minimum several minutes before deploying to Kubernetes.** The 4–15 s startup overhead (local Kind, pre-pulled images, growing with concurrency) is modest for most pipeline workloads. In production clusters with cold image pulls, overhead may be higher (10–60 s); pre-warming images is recommended. Short-duration Dagster ops should be batched into longer jobs when targeting a Kubernetes executor.
4. **Account for Kubernetes startup overhead in SLA calculations.** Pipeline SLAs should add 4–60 s for container startup when running on Kubernetes, depending on concurrency level and whether images are pre-pulled (local/pre-warmed) or cold-pulled (production registry). This overhead grows sub-linearly with concurrency due to concurrent initialisation contention.
5. **Test blast radius containment under sustained memory pressure, not just nominal load.** Both environments successfully contained blast radius at moderate concurrency. The real containment advantage of Kubernetes emerges at extreme load ($> L7$) where VM-level OOM kills can affect multiple containers simultaneously. Test at your highest expected concurrency before relying on

containment assumptions.

6. **Consider the operational cost of Kubernetes.** Kubernetes requires significantly more operational expertise than a single-VM Docker deployment. For teams running fewer than 5–7 concurrent Dagster jobs, the VM offers competitive performance with substantially lower operational overhead.

6.4 Future Research

Several directions emerge from the findings and limitations of this study:

1. **Multi-node Kubernetes cluster.** This study used a single-node Kind cluster. A multi-node cluster would introduce real network scheduling decisions, pod placement policies, and inter-node resource contention. Future work should replicate these experiments on a 3-node or 5-node cluster to separate single-host scheduling effects from true distributed orchestration overhead.
2. **Heterogeneous workload types.** The workload in this study was deliberately homogeneous (CPU-bound SHA-256 + fixed memory allocation). Real Dagster pipelines include I/O-heavy ops (database queries, S3 reads), memory-intensive transformations (large DataFrame joins), and network-bound steps. Future work should characterise the crossover point across a taxonomy of workload types.
3. **Job duration sensitivity analysis.** Container startup overhead ($\approx 4\text{--}15\text{ s}$ in the local Kind environment, growing with concurrency) is the main cost for short-duration jobs. A systematic sensitivity analysis varying job duration from 30 s to 30 min would produce a two-dimensional crossover surface (concurrency $\times$ job duration) that gives a more complete decision framework.
4. **Production Kubernetes scheduler comparison.** This experiment used the default Kubernetes scheduler. Production clusters often use custom scheduling policies (bin packing, spread scheduling, priority classes). Future work should quantify how scheduler policy affects the crossover point.
5. **Higher VM memory configuration.** The reliability crossover in this study occurred early (L3) because the 4 GB VM ran out of memory with three concurrent 400 MB containers plus the infrastructure stack. Repeating the experiments on an 8 GB or 16 GB VM would raise the OOM threshold and expose a performance-only crossover regime where reliability is not the binding constraint, giving a cleaner separation of overhead and contention effects.

6. **Managed cloud Kubernetes.** Repeat experiments on GKE, EKS, or AKS to validate whether the crossover point shifts in a managed multi-node environment with production scheduling policies and auto-scaling.
7. **Cross-framework comparison.** Compare crossover points across different workflow orchestration frameworks (Airflow, Prefect, Argo Workflows) to determine whether the Dagster-specific startup overhead is representative or an outlier.

Bibliography

- Jatin Arora. *Shared Resource Contention-Aware Schedulability Analysis of Hard Real-Time Systems*. PhD thesis, CISTER Research Centre, Polytechnic Institute of Porto, 2023.
- Emiliano Casalicchio. A study on performance measures for auto-scaling CPU-intensive containerized applications. *Cluster Computing*, 22:995–1006, 2019. doi: 10.1007/s10586-018-02890-1.
- Bongmin Choi, Joongheon Park, Changhyeon Lee, and Dongsu Han. pHPA: A proactive autoscaling framework for microservice chain. In *APNet 2021: Proceedings of the 5th Asia-Pacific Workshop on Networking*, 2021. doi: 10.1145/3469393.3469401.
- Ivan Čilić, Petar Krivić, Ivana Podnar Žarko, and Mario Kušek. Performance evaluation of container orchestration tools in edge computing environments. *Sensors*, 23(8):4008, 2023. doi: 10.3390/s23084008.
- Hengyi Li, Weixiong Rao, Baocheng Hu, Yuan Tian, and Jianjun Shen. Energy-aware elastic scaling algorithm for microservices in Kubernetes clouds. *Journal of Network and Computer Applications*, 242:104218, 2025. doi: 10.1016/j.jnca.2025.104218.
- Qianli Liu, Yiyu Yang, Dong Du, Yubin Xia, Peng Zhang, Jianzhong Feng, James R. Larus, and Haibo Chen. Harmonizing efficiency and practicability: Optimizing resource utilization in serverless computing with Jiagu. In *2024 USENIX Annual Technical Conference*, 2024.
- Ashwini K. Nanda, Hong Shing, Ten-Hwang Tzen, and Lionel M. Ni. Resource contention in shared-memory multiprocessors: A parameterized performance degradation model. *Journal of Parallel and Distributed Computing*, 12:313–328, 1991.
- Thanh-Tung Nguyen, Young-Jin Yeom, Taehong Kim, Dae-Heon Park, and Sehan Kim. Horizontal pod autoscaling in Kubernetes for elastic container orchestration. *Sensors*, 20(16):4621, 2020. doi: 10.3390/s20164621.
- Manh-Nhat Tran, Duc-Dung Vu, and Younghan Kim. A survey of autoscaling in Kubernetes. In *International Conference on Ubiquitous and Future Networks (ICUFN)*, 2022.
- Hao Yuan and Shun Liao. A time series-based approach to elastic Kubernetes scaling. *Electronics*, 13(2):285, 2024. doi: 10.3390/electronics13020285.

Appendices

Appendix A

Kind Cluster Configuration

```
1 # kind-config.yaml
2 kind: Cluster
3 apiVersion: kind.x-k8s.io/v1alpha4
4 nodes:
5   - role: control-plane
6   - role: worker
7     kubeadmConfigPatches:
8       - |
9         kind: JoinConfiguration
10        nodeRegistration:
11          kubeletExtraArgs:
12            system-reserved: "cpu=500m,memory=512Mi"
13            kube-reserved: "cpu=500m,memory=512Mi"
```

Listing A.1: Kind cluster configuration (kind-config.yaml).

Appendix B

Dagster Helm Values (Kubernetes Run Launcher)

```
1 # values.yaml
2 dagster:
3   runLauncher:
4     type: K8sRunLauncher
5     config:
6       k8sRunLauncher:
7         jobNamespace: dagster
8         resources:
9           requests:
10             cpu: "500m"
11             memory: "512Mi"
12           limits:
13             cpu: "1000m"
14             memory: "1Gi"
```

Listing B.1: Dagster Helm values for Kubernetes Run Launcher (values.yaml).

Appendix C

Test Harness Pseudocode

```
1 import concurrent.futures
2 import time
3 import requests # for Dagster GraphQL API
4
5 DAGSTER_GRAPHQL_URL = "http://localhost:3000/graphql"
6
7 def submit_run(job_name, run_config):
8     """Submit a single Dagster run via GraphQL API."""
9     mutation = """
10     mutation LaunchRun($input: LaunchRunInput!) {
11         launchRun(input: $input) {
12             run { runId status }
13         }
14     }
15     """
16     # ... submit and return run_id, submission_timestamp
17     pass
18
19 def poll_run_status(run_id):
20     """Poll until run completes or fails.
21     Return final status + timestamps."""
22     # ... poll GraphQL API for run events
23     pass
24
25 def run_experiment(concurrency_level, job_name, run_config):
26     """Submit N runs concurrently and collect all metrics."""
27     results = []
28     with concurrent.futures.ThreadPoolExecutor(
29         max_workers=concurrency_level
30     ) as pool:
31         futures = [
32             pool.submit(submit_run, job_name, run_config)
33             for _ in range(concurrency_level)
```

```
34     ]
35     run_ids = [f.result() for f in futures]
36
37     # Poll all runs to completion
38     for run_id in run_ids:
39         result = poll_run_status(run_id)
40         results.append(result)
41
42     # Export to CSV
43     export_to_csv(results, f"exp_{concurrency_level}.csv")
44     return results
```

Listing C.1: Test harness pseudocode for concurrent Dagster run submission.

Appendix D

Experiment Data Collection Template

Table D.1: Raw data collection template for experimental runs (VM environment, one repetition shown; repeat for reps 2 and 3 and for K8s environment). Pod scheduling and container startup columns are K8s-only; enter “—” for VM rows.

Rep	Level	Env	SR (%)	Mean (s)	Std (s)	CPU (%)	Mem (MB)	Sched (s)	Start (s)
1	L1	VM						—	—
1	L1	K8s							
1	L2	VM						—	—
1	L2	K8s							
1	L3	VM						—	—
1	L3	K8s							
1	L5	VM						—	—
1	L5	K8s							
1	L7	VM						—	—
1	L7	K8s							
1	L10	VM						—	—
1	L10	K8s							
<i>Repeat for repetitions 2 and 3.</i>									